Agent-Computer Observation Interfaces Enable Dynamic Computer Use

Anonymous Authors
https://anonymous.4open.science/r/aoi-anonymous

May 17, 2026

Abstract

The interface, not the model. SWE-agent [Yang et al., 2024] showed that the agent-computer *action* interface is an underexplored design axis for software-engineering agents. We show the same for the *observation* interface in computer-use (CU) agents: getting it right unlocks dynamic computer use from any existing CU model without retraining. Current CU agents—closed and open-source alike—observe the world through periodic screenshots every 3–5 s and have no audio perception, leaving them blind to video playback, animations, transient UI events, meetings, notifications, and spoken instructions. We introduce the **Agent Observation Interface (AOI)**, a model-agnostic perception layer that decouples observation (continuous, adaptive) from action (discrete). Its three gated components—inter-step keyframe capture, volume-gated audio scene understanding, and CU-model-generated visual narration that persists as text—produce nothing on static, silent content, reducing the AOI to the standard loop.

Results. We introduce **DynaCU-Bench**: 100 dynamic browser tasks across 10 categories plus a 50-task static set for zero-overhead verification. Six CU models—Claude Sonnet 4.6, GPT-5.4, Gemini 2.5 Flash, Grok-4, EvoCUA-32B, Fara-7B—gain +17 to +48 pp over their screenshot-only baselines (paired McNemar $p < 10^{-3}$ for every model); the strongest reaches 82% on Claude Sonnet 4.6 (seed 1; 3-seed mean 78.7%, std 3.1 pp; §6.3). On a 12-task audio-focused subset, AOI full (12/12) beats two production streaming baselines—OpenAI Realtime [OpenAI, 2024] (3/12) and Gemini Live [Google, 2025] (0/12)—both adapted to drive the same browser; an adapter sanity check rules out infrastructural failure.

Two surprises about the observation interface. *Selection doesn't matter, capture does*: four inter-step keyframe selection strategies (uniform, random, pixel-difference, CLIP) are statistically indistinguishable (pairwise McNemar $p > 0.5$, ∼58% each), replicated on the open-source Qwen3-VL-32B vision model. ASR adds +6 pp, and the largest single-component gain comes from *persistent text memory*: a narration-discarded ablation isolates a significant +8 pp contribution from retaining narration after keyframes are pruned ($p = 0.039$ at N=100), plus a directional +10 pp from inference-time chain-of-thought during narration ($p = 0.12$; confirming at $p < 0.05$ would require N∼300). A `standard_structured` control decomposes each cloud model's gain into prompt-format and perception components; for models predating Gemini 3 both contribute positively (Claude: +25 pp perception vs. +19 pp prompt; Gemini 2.5 Flash: +23 vs. +25). On Gemini 3 Flash the headline residual collapses to +9 pp ($p = 0.18$). A four-way decomposition (`standard`/`standard_audio`/`aoi_audio`/`aoi_full`) shows this is not internalised perception but sign-separated component effects: audio +12 pp, structured scaffold +9 pp (bi-directional per category), keyframes −**12** pp. Replacing the default bundle with `aoi_audio` (no keyframes) reaches 57/100, a +21 pp recovery and +12 pp above `aoi_full`. Implication: the observation interface is a separable design axis along which models converge unevenly *and unevenly per component*. Keyframe imagery is the first AOI element we observe crossing from net-positive to net-negative across model generations, so component selection must become per-model rather than a fixed bundle.

1 Introduction

The agent-computer observation interface as a design axis. SWE-agent [Yang et al., 2024]'s core contribution was not a new model but a careful design of the agent-computer *action* interface—which commands the model is given, how inputs and outputs are formatted, what feedback they return. Small changes there dramatically affect what a fixed model can accomplish, making the action interface a separable, underexplored design axis. We make the analogous argument for the *observation* interface in computer-use (CU) agents. Every CU agent we are aware of—closed-source [Anthropic, 2024a, OpenAI, 2025, Google DeepMind, 2025, Andreux et al., 2025] and open-source [Wang et al., 2025a,b, Awadallah et al., 2025, Song et al., 2025]—ties observation to action: one screenshot per action step. We argue that decoupling observation (continuous, adaptive) from action (discrete) is the single change that unlocks dynamic computer use from any existing CU model with zero retraining.

The current state. On OSWorld [Xie et al., 2024], screenshot-based agents have moved from sub-10% at introduction to over 50% in 2025 (Surfer 2 [Andreux et al., 2025]: 50.7%; UI-TARS-2 [Wang et al., 2025a]: 47.5%), closing most of the gap to the 72.4% human baseline. These agents can fill forms, navigate websites, edit documents, and use professional software.

The blind spot. All these agents observe through *discrete static screenshots* every 3–5 seconds, and are entirely deaf. Between screenshots, they are blind. They cannot understand on-screen video, catch transient UI events that auto-dismiss between observations, hear meeting speech or notification sounds, or perceive animations. A comprehensive survey [Sager et al., 2025] catalogs many open challenges but does not identify the observation modality as a gap. Quantitative evidence is mounting: GUI-World [Chen et al., 2024] shows video LLMs fail on all temporal GUI tasks; VideoWebArena [Jang et al., 2024] reports that long-context models perform *worse* with video than without; VideoGameBench [Zhang et al., 2025] finds even paused-game evaluation yields only 1.6% completion; CUA-Suite [Chen et al., 2025] catalogs 55 hours of temporal dynamics that screenshot-based agents miss entirely.

Why existing approaches fall short. Retraining CU models on video data is expensive, model-specific, and undemonstrated at scale. Video-native models (Gemini) amount to brute-force frame sampling that wastes tokens on redundant static frames. Native real-time multimodal models (Gemini Live [Google, 2025], OpenAI Realtime [OpenAI, 2024]) work but lock users into a single provider, offer no selective perception, and require API migration; we evaluate them in Section 7.6. Cradle [Tan et al., 2024]'s pixel-level frame differencing is the closest predecessor but lacks audio perception, visual narration, and a structured observation record.

Our approach. The **Agent Observation Interface (AOI)** is a lightweight perception layer that converts continuous screen and audio streams into the sparse images and text that existing CU models already accept. It sits between the environment and any image-based CU model; on static, silent content its gates produce nothing and behaviour reverts to the standard loop; on dynamic content it produces extra keyframes, audio transcripts, and CU-model-generated narration that persists as text after the source images are pruned.

Contributions.

1. We identify the **observation interface** as a critical but overlooked CU design axis, complementary to SWE-agent's action interface, and articulate *decoupling observation from action* as the operative design principle.
2. We introduce the **AOI**, a model-agnostic perception layer with three gated components (inter-step keyframe capture, volume-gated audio scene understanding, CU-model-generated visual narration).
3. We release **DynaCU-Bench** (100 dynamic tasks) and a **Static-50** no-degradation baseline.

(A cross-engine ASR check with `espeak` and real asciinema casts appears in Appendix E as a no-degradation pilot, not a perception comparison.)

4. Six CU models equipped with the AOI and **zero retraining** gain +17 to +48 pp on dynamic tasks (paired McNemar $p < 10^{-3}$ for every model).
5. A controlled *narration-discarded* ablation decomposes the visual-narration component's +18 pp gain into a significant +8 pp from *persistent text memory* ($p = 0.039$) plus a directional +10 pp from inference-time chain-of-thought during narration generation ($p = 0.12$ at N=100).
6. Two confounds are ruled out: a `standard_structured` run on DynaCU-Bench separates prompt-format from perception (Section 7.5), and an open-source Qwen3-VL-32B replication confirms the selection-method convergence is mechanism-driven, not Claude-specific (Section 6.9).
7. We benchmark against monolithic streaming baselines (Gemini Live, OpenAI Realtime) with an adapter sanity check (Section 7.6, Appendix F).

2 Background: The CU Agent Loop

All current CU agents follow the same observe–reason–act loop:

$$\texttt{repeat:} \quad s_t \leftarrow \text{Screenshot}(), \quad a_t \leftarrow \text{CU\_Model}(s_t, \tau), \quad \text{Execute}(a_t), \quad \text{Wait}(\delta) \tag{1}$$

where τ is the trajectory history and δ is a post-action buffer (typically 500 ms). The interval between observations is determined by model inference time + action execution + buffer, typically 3–5 seconds total. Everything that occurs between screenshots—visual changes, audio events, transient UI elements—is invisible to the agent.

Some models support multi-image input through screenshot history (UI-TARS: up to 5 images, Claude CU: ∼10), but all images are post-action screenshots from prior steps. None captures what happens *between* steps.

Four categories of content are systematically missed: (i) *transient UI events*—toast notifications and dialogs that appear and vanish within one observation interval; (ii) *continuous visual media*—video playback, animated tutorials, transitioning slides; (iii) *audio events*—meeting speech, notification sounds, error alerts; (iv) *periodic visual noise*—loading spinners and blinking cursors that should be *ignored* but trigger false positives in naive frame differencing.

3 System Design

The AOI sits between the environment and any existing CU model, observing the entire interval between agent steps and providing the model with additional keyframes, audio descriptions, and accumulated text context—all in the standard image + text format that every CU model already accepts. For static, silent tasks, the AOI produces nothing and behavior is identical to the standard loop; for dynamic tasks, the model receives strictly more information. No retraining is needed.

3.1 Architecture Overview

The AOI has three components, each preceded by a fast gate (sub-millisecond) that short-circuits processing when there is nothing to perceive. For a frame that passes the gates, the additional cost is dominated by CLIP-ViT-B/16 (∼7 ms on GPU) for visual frames and Whisper large-v3 (variable, on demand) for audio segments; for the static, silent frames that dominate most workloads the per-frame cost is the sub-millisecond gate alone.

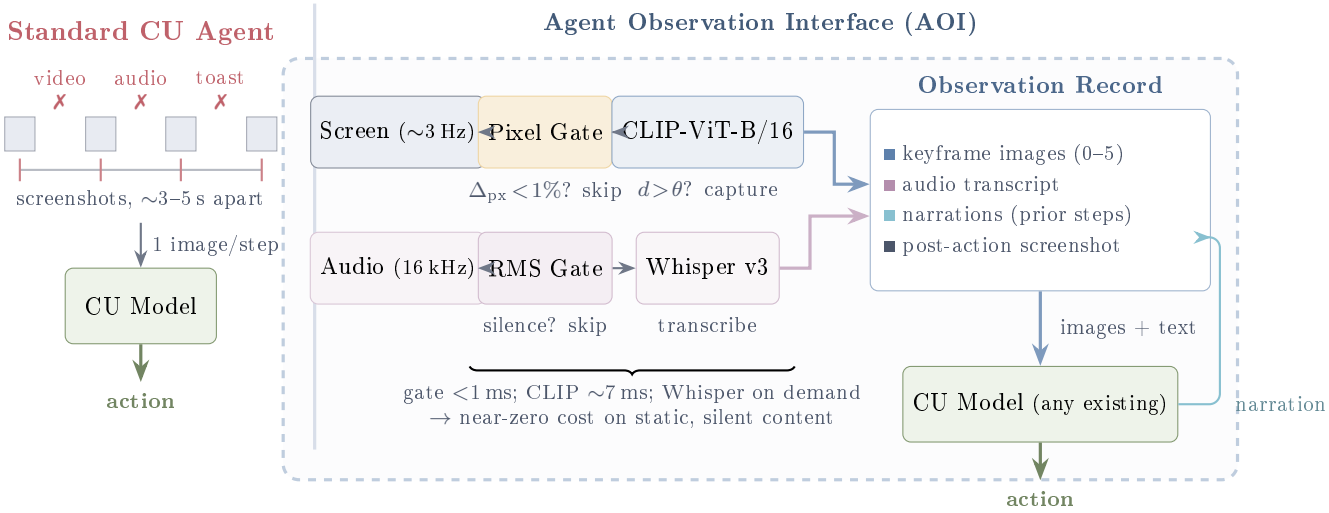

Figure 1: **Standard CU agents vs. AOI-equipped agents.** *Left:* Current agents observe through periodic screenshots (∼3–5 s apart) and are blind and deaf between observations—missing video, audio, and transient UI events (✗). *Right:* The AOI continuously monitors screen and audio streams through fast gates (<1 ms) that skip processing for static, silent content. When gates fire, keyframe extraction (shown here with optional CLIP filtering; see Section 6.9) and Whisper ASR produce images and text for the observation record. Visual narrations feed back as persistent text memory. The AOI wraps any existing CU model with zero retraining.

(1) **Inter-step keyframe capture**: continuous screen sampling at ∼3 Hz with gating to suppress redundant frames. Produces 0–5 keyframe images per step.
(2) **Volume-gated audio observer**: RMS energy gate followed by ASR (Whisper large-v3). Produces a text transcription per step, only when audio is present.
(3) **Visual narration context**: the CU model itself generates a brief text description of new visual information as a side-output of each inference call. These narrations accumulate in the trajectory and persist after keyframe images are pruned from context.

3.2 Inter-Step Keyframe Capture

Between agent steps, the screen may change (a video plays, a dialog appears, a slide transitions) or stay static. The AOI continuously samples the screen at ∼3 Hz and selects a subset of frames to include as keyframe images in the observation record (up to 5 per step). Multiple selection strategies are possible—uniform fixed-rate sampling, pixel-level differencing, random sampling, or semantic filtering via CLIP embeddings [Radford et al., 2021]. We evaluate all four in Section 6.9 and find that all converge to similar accuracy, so the default implementation uses simple pixel-change gating: if fewer than 1% of pixels changed since the last captured frame, the sample is skipped (cost: <1 ms on CPU). For static screens, no keyframes are produced and the component has zero overhead. Algorithm 1 formalises the two-stage CLIP variant; the default selector drops the CLIP stage (lines 5–10) and uses only Stage 1.

3.3 Volume-Gated Audio Observation

Current CU agents have zero audio perception. They cannot hear meeting speech, notification sounds, or error alerts. Pure ASR models (Whisper) transcribe speech but produce nothing for

Algorithm 1 Two-stage adaptive keyframe extraction (CLIP variant). This algorithm is shown for completeness; the selection-method ablation in Section 6.9 finds it statistically indistinguishable from the simpler pixel-only gate (Stage 1) and uniform / random sampling, so the default selector in every main result is Stage 1 alone.

Require: Screen sample x_t, anchor embedding e_{anchor}, pixel threshold α, CLIP threshold θ

$\Delta_{\text{px}} \leftarrow \text{PixelChangeRatio}(x_t, x_{\text{prev}})$
if $\Delta_{\text{px}} < \alpha$ **then**
  return SKIP {Stage 1: no pixel change}
end if
$e_t \leftarrow \text{CLIP}(x_t)$
$d \leftarrow 1 - \cos(e_t, e_{\text{anchor}})$
if $d > \theta$ **then**
  $e_{\text{anchor}} \leftarrow e_t$ {Re-anchor}
  return CAPTUREKEYFRAME(x_t, t)
end if
return SKIP {Stage 2: below semantic threshold}

non-speech audio events.

Volume gate. In most computer use—file editing, form filling, silent web browsing—there is no audio. Computing RMS energy of the audio buffer is sub-millisecond. If below the silence threshold, the audio model call is skipped entirely. Cost: ∼0 ms for the majority of steps.

When audio is present, we use Whisper large-v3 [Radford et al., 2023] for speech transcription, operating on 16 kHz mono audio captured from the system audio output via PulseAudio virtual devices. The audio buffer spans the full inter-step interval including the post-action buffer, capturing action feedback sounds (click confirmations, error beeps).

Overlapping windows. Agent step boundaries are determined by model inference latency, not by speech content. A typical 3.5-second audio chunk captures ∼8–9 words at normal speaking rate, almost always falling mid-sentence. To resolve boundary artifacts, the audio buffer includes ∼3.5 seconds of overlap from the previous interval, providing acoustic continuity across step boundaries.

3.4 Visual Narration for Long-Term Context

CU models retain only the most recent images in context. Keyframes from earlier steps are pruned. For tasks spanning many steps, the agent loses all visual memory of earlier content. Audio transcriptions persist naturally as text in the trajectory, but visual observations have no analogous persistence.

At each step, the CU model outputs both an action and a brief visual narration—a text description of what is visually new in the current keyframes. This narration is generated in the same inference call that produces the action (zero additional compute) and persists indefinitely in the trajectory, even after the corresponding images are pruned. Following the caption-then-reason principle of LLoVi [Zhang et al., 2023], narrations are generated by the CU model itself (no separate captioner) and are task-relevant rather than generic.

3.5 Observation Record

At each step, the CU model receives a structured document combining text context from recent prior steps (audio transcriptions, visual narrations, actions taken) and raw observations from the current

Observation Record — Step N (Meeting Task B-E1)

```
# CONTEXT -- text from prior steps (no images)
Step N-1 (t ≈ 18.5-22.0 s):
AUDIO: "Here's the team.  As you can see we
have strong cross-functional coverage."
VISUAL: "Meeting slide shows Project Team
with 6 members listed in a grid."
ACTION: wait()

# NEW -- current interval observations
Step N (t ≈ 22.0-25.5 s):
AUDIO: "And I want to mention, we've moved
the launch date to April 28th.
Please update your calendars."
[IMAGE: post-action screenshot]

# TASK
Read the meeting slides and listen to the
discussion.  What is the new launch date?
Enter it in the Launch Date field.
```

Figure 2: Example observation record sent to the CU model at step N of a meeting task. The **context** section provides text from prior steps (audio transcriptions and visual narrations persist after images are pruned). The **new** section contains the current audio transcription and the post-action screenshot (image). In this case, no keyframes were captured (the slide did not change), so only the screenshot is included as an image. The task instruction is appended at the end.

interval (new audio transcription, any captured keyframe images, and the post-action screenshot). For static, silent tasks, this document contains only the post-action screenshot—identical to the standard loop. Figure 2 shows a concrete example.

4 Implementation

The AOI is implemented in Python (∼2,600 lines) as a modular layer that wraps any CU model. Screen capture uses a headless browser (Playwright) at ∼3 Hz. Audio capture and injection use PulseAudio virtual devices (virtual speaker and monitor source) at 16 kHz mono. CLIP-ViT-B/16 runs on GPU (∼150 MB VRAM). Whisper large-v3 runs as a separate HTTP service on CPU to avoid GPU contention with the CU model. Audio synthesis for benchmark tasks uses the Web Speech API (`speechSynthesis`).

CU models are accessed through their native APIs. Cloud models (Claude Sonnet 4.6, GPT-5.4, Gemini 2.5 Flash) use HTTP APIs. Local models (EvoCUA-32B, Fara-7B) are served via vLLM with 4-bit quantization on a single NVIDIA RTX PRO 6000 (96 GB VRAM).

5 DynaCU-Bench

5.1 Design Principles

Existing CU benchmarks [Xie et al., 2024, Zhou et al., 2023, He et al., 2024] consist almost entirely of tasks solvable from static screenshots. We introduce **DynaCU-Bench**, a benchmark of 100 browser-based tasks that specifically require dynamic visual and/or audio perception—tasks that are unsolvable from static screenshots alone.

Table 1: DynaCU-Bench task categories. A = audio perception, V = visual-temporal perception, I = real-time interaction. Each category contains 3 easy, 4 medium, and 3 hard tasks.

Cat.	Domain	Axes	Description
A	Podcast comprehension	A	Listen to audio narration, extract facts or answer questions
B	Meeting participation	A+V+I	Follow meeting with slides and speech, take notes or act
C	Video/screencast	V	Watch terminal recordings or IDE demos with auto-advancing frames
D	Carousel/animation	V	Perceive CSS transitions, rotating content, product carousels
E	Live dashboard	V	Monitor real-time updating dashboards with `setInterval` changes
F	Transient UI events	V	Respond to toast notifications, auto-dismissing banners
G	Phone/voice call	A+I	Listen to `speechSynthesis` calls and respond
H	Interview/voice	A+I	Answer questions via `SpeechRecognition`, complete interviews
I	Collaborative editing	V+I	Handle remote changes in collaborative document editors
J	Browser games	V+I	Play interactive games requiring temporal attention

Design principles: (i) tasks must *require* temporal visual or audio perception; (ii) tasks represent realistic computer-use scenarios, not artificial constructs; (iii) tasks execute in a reproducible headless browser environment; (iv) tasks cover all four dynamic content categories from Section 2; (v) tasks include difficulty stratification (3 easy, 4 medium, 3 hard per category).

5.2 Task Categories

DynaCU-Bench spans 10 categories across three capability axes: audio perception (A), visual-temporal perception (V), and real-time interaction (I). Each category has 10 tasks.

5.3 Evaluation Protocol

Each task executes in a headless Chromium browser (Playwright) with PulseAudio virtual devices for audio I/O. Audio content is delivered via the Web Speech API (`speechSynthesis`) with edge-TTS for high-quality voices. Agent audio input is injected into a virtual microphone via `pacat`.

Three evaluation modes are used: *DOM-based* (93 tasks): a deterministic JavaScript function `getTaskResult()` returns the expected value; *LLM-judged* (1 task): a judge model scores the agent's response against a rubric; *Hybrid* (6 tasks): DOM gate (did the agent act?) combined with LLM quality assessment. Tasks have a maximum of 15 agent steps and a per-task wall-clock budget of `duration_s + 10 s + AOI overhead`, where `duration_s` is the task's stimulus duration (typically 15–90 s, scaling with difficulty); whichever limit is reached first ends the run. These wall-clock budgets matter only for the slowest models (e.g. Grok-4 at ∼80–180 s/call; Section 6.4).

Table 2: DynaCU-Bench results: task success rate (%) on the 100 dynamic tasks. Brackets show 95% Wilson confidence intervals. **Bold** indicates the best result per model. Δ is the absolute gain over the standard baseline; p-values are paired McNemar's exact mid-p tests. All ablation modes are tested on Claude Sonnet 4.6. "Claude 4.6" abbreviates Claude Sonnet 4.6 throughout. Grok-4 is the slowest model at inference (∼80–180 s/call), which holds its absolute scores down even when AOI unblocks perception; see Section 6.4.

Configuration	Claude 4.6	GPT-5.4	Gemini 2.5	Grok-4	EvoCUA-32B	Fara-7B
Standard (screenshot)	38 [29.1, 47.8]	37 [28.2, 46.8]	21 [14.2, 30.0]	4 [1.6, 9.8]	18 [11.7, 26.7]	17 [10.9, 25.5]
Uniform 1 FPS	58 [48.2, 67.2]	—	—	—	—	—
Uniform 3 FPS	56 [46.2, 65.3]	—	—	—	—	—
Pixel-diff only	58 [48.2, 67.2]	—	—	—	—	—
Random keyframes	56 [46.2, 65.3]	—	—	—	—	—
AOI (visual only)	58 [48.2, 67.2]	—	—	—	—	—
AOI (visual + ASR)	64 [54.2, 72.7]	—	—	—	—	—
AOI (full)	**82** [73.3, 88.3]	**57** [47.2, 66.3]	**69** [59.4, 77.2]	**25** [17.5, 34.4]	**55** [45.2, 64.4]	**34** [25.5, 43.7]
Δ (full − std)	+44	+20	+48	+21	+37	+17
p (McNemar)	1.3×10^{-10}	8.8×10^{-5}	2.9×10^{-12}	3.0×10^{-6}	3.0×10^{-9}	4.9×10^{-4}

6 Evaluation

6.1 Setup

CU models. We evaluate six models spanning the capability and scale spectrum: (i) Claude Sonnet 4.6 [Anthropic, 2024a] (closed-source); (ii) GPT-5.4 [OpenAI, 2026] (closed-source; the latest GPT-5 series tool-calling-capable vision model exposed by OpenAI's chat-completions API at evaluation time); (iii) Gemini 2.5 Flash [Google DeepMind, 2025] (closed-source); (iv) Grok-4 [xAI, 2026] (closed-source, xAI; reported in Section 6.4 with attention to the inference-latency confound); (v) EvoCUA-32B [Meituan, 2026] (open-source, 32B; weights and inference recipe released through Meituan's GitHub); (vi) Fara-7B [Awadallah et al., 2025] (open-source, 7B). We use each model's vendor-recommended decoding settings (temperature, top-p) unchanged. We do not rely on any vendor-reported OSWorld score for any claim in this paper; we report only the numbers measured under our own DynaCU-Bench harness.

Observation configurations. We compare 8 observation modes:

(1) **Standard**: one screenshot per step, no audio (current paradigm baseline).
(2) **Uniform 1 FPS**: capture at fixed 1 FPS, feed recent N frames.
(3) **Uniform 3 FPS**: capture at fixed 3 FPS, feed recent N frames.
(4) **Pixel-diff**: capture when pixel change ratio exceeds threshold, no CLIP filtering.
(5) **Random keyframes**: randomly sample the same number of frames as our method.
(6) **AOI (visual only)**: adaptive keyframes (CLIP-based), no audio.
(7) **AOI (visual + ASR)**: adaptive keyframes + Whisper speech transcription.
(8) **AOI (full)**: adaptive keyframes + Whisper ASR + visual narration.

Ablation configurations (2)–(7) are evaluated with Claude Sonnet 4.6; the full AOI and standard baseline are evaluated with all six models.

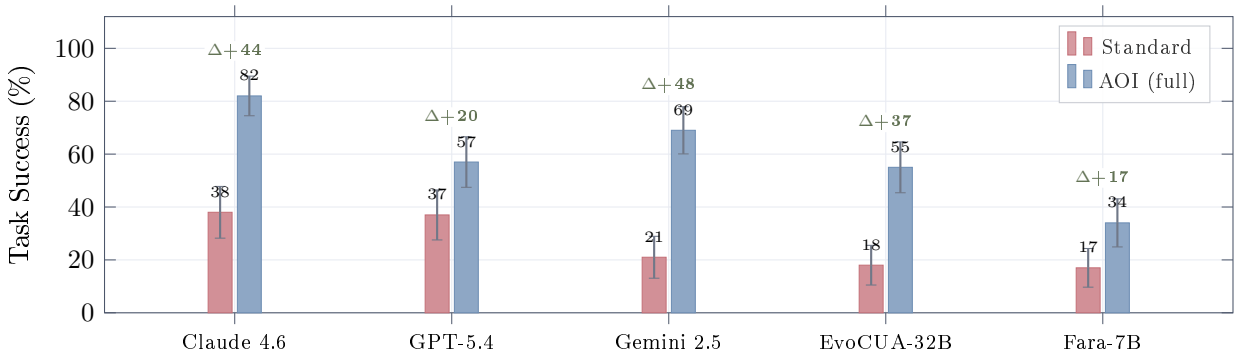

Figure 3: **Main results on DynaCU-Bench (100 tasks).** The AOI produces large gains across CU models without any retraining. Green numbers show absolute improvement in percentage points. Gemini 2.5 Flash shows the largest absolute gain (+48 pp) and relative gain (3.3×), while Claude achieves the highest absolute score (82%, seed 1; 3-seed mean 78.7%). Five of the six models tested are plotted here; Grok-4 (standard 4, AOI full 25, Δ+21 pp) is omitted from the chart for legibility and reported separately in Table 4 (Section 6.4). Error bars are approximate 95% intervals using a symmetric Gaussian span; exact (asymmetric) Wilson intervals are tabulated in Table 2 and used for all reported p-values.

6.2 Main Results

Table 2 and Figure 3 present the main results. The AOI produces large, consistent gains across all six models, all highly significant by paired McNemar's exact test ($p < 10^{-3}$ for every model; $p < 10^{-9}$ for Claude and Gemini, $p < 10^{-8}$ for EvoCUA). *Note on statistical power:* Each configuration is evaluated once on 100 binary-outcome tasks (no repeated trials). We use paired McNemar's exact mid-p tests on the discordant tasks to assess significance and report 95% Wilson confidence intervals on the marginal success rates. Pairwise differences between selection methods (e.g., 56% vs. 58%) are not significant ($p > 0.5$ for every pair, see Table 10); the large between-tier gaps are.

Claude Sonnet 4.6 improves from 38% to 82% (+44 pp) with the full AOI. This is the strongest absolute result. The ablation modes reveal that all inter-step visual capture methods converge to 56–58% (a +18–20 pp gain from additional frames alone, regardless of selection method), adding ASR reaches 64%, and visual narration provides the largest single-component gain, reaching 82%. Section 7.2 decomposes this +18 pp gap into a significant +8 pp persistent-memory component plus a directional, underpowered +10 pp inference-time CoT component—only the first reaches significance at N=100, but the sign and magnitude of the second are consistent with a real CoT effect that would require ∼300 tasks to confirm.

GPT-5.4 improves from 37% to 57% (+20 pp). The gain is consistent with Claude's but the absolute ceiling is lower, suggesting GPT-5.4's CU capabilities are the binding constraint once perception is adequate.

Gemini 2.5 Flash scores 21% on the standard baseline despite being a video-native model with built-in streaming capabilities. This confirms that video-native architectures do not automatically solve the observation problem—the standard CU agent loop still constrains Gemini to periodic screenshots. With the AOI, Gemini improves to 69% (+48 pp)—the largest absolute gain of any model tested. Section 7.5 (Table 17) decomposes this gain: roughly half (+25 pp) comes from the AOI's structured prompt format itself rather than from new perception (+23 pp), so the right interpretation is that a video-native model benefits substantially from *both* better presentation of existing context and from explicit between-step perception.

Task B-E1: Meeting — "What is the new launch date?"

Standard (screenshot-only) — FAIL

15 steps, 51.2 s

1. `wait()` — sees static slide
2. `wait()` — same slide
3. `wait()` — same slide
4. `wait()` — same slide
5. `wait()` — same slide

⋮

15. `fill(#launchDate, "TBD")` × guesses

Agent never hears the spoken date.
Waits 15 steps, then guesses.

AOI (full) — PASS

2 steps, 26.2 s

1. `wait()`
   Audio: "Here's the team. As you can see we have strong cross-functional coverage."
   Narration: "Meeting slide shows Project Team with 6 members."
2. `fill(#launchDate, "April 28th")` ✓
   Audio: "We've moved the launch date to April 28th. Please update."
   Narration: "Audio announced the new launch date."

Agent hears "April 28th" and fills the form immediately.

Task C-E1: Video — "What was the first command in the terminal recording?"

Standard — PASS (6 steps)

1. `wait()` — recording playing
2. `wait()` — missed first command
3. `wait()` — scrolling through
4. `wait()`
5. `wait()`
6. `fill(...)` ✓ lucky timing

AOI — PASS (1 step)

1. `fill(#answerInput, "git clone ...")` ✓
   Narration: "The terminal recording shows the first command was git clone..."

Narration captures the command from the recording immediately.

Figure 4: Trajectory comparison on two representative tasks (*illustrative; cherry-picked to expose the mechanism*). **Top:** Meeting task (B-E1) where the launch date is spoken aloud. The standard agent cannot hear it and guesses after 15 futile `wait()` steps. The AOI agent transcribes the audio, hears "April 28th," and acts in 2 steps. **Bottom:** Video task (C-E1) where the answer appears in a terminal recording. The standard agent needs 6 steps with lucky timing; the AOI agent's narration captures the content in 1 step.

EvoCUA-32B improves from 18% to 55% (3× improvement, +37 pp). This open-source 32B model, which barely functions on dynamic tasks without the AOI, becomes competitive with GPT-5.4's AOI-equipped performance.

Fara-7B improves from 17% to 34% (+17 pp), doubling its score. As the smallest model (7B parameters), it is most constrained by reasoning capacity, but still benefits substantially from improved perception.

6.3 Variance: 3-Seed Replication of the Headline

Each Table 2 cell is one trial of 100 binary tasks at the model's default sampling temperature, folding decoding variance into "task hardness" without separating the two. To quantify the decoding component, we re-run Claude Sonnet 4.6 + AOI full for two additional seeds using the same harness; results are in Table 3.

The decoding-variance question is independent of the McNemar statistical-power question: Mc-

Table 3: Three-seed replication of the headline Claude Sonnet 4.6 AOI full configuration on the 100 dynamic tasks. Seed 1 is the run reported in Table 2; seeds 2–3 are fresh re-runs at the API's default sampling parameters (Anthropic's Messages API does not expose a deterministic seed, so each call samples fresh). The 82% headline is at the upper edge of a 76–82% range, with 3-run mean 78.7% and standard deviation 3.1 pp. The McNemar tests in Table 2 compare within-seed paired outcomes and remain valid for the standard-vs.-AOI contrast; the seed std reported here is the additional uncertainty on the AOI full absolute score.

Configuration	**Seed 1**	**Seed 2**	**Seed 3**	**Mean**	**Std**
Claude Sonnet 4.6 + AOI full	82	78	76	78.7	3.1

Table 4: Grok-4 on DynaCU-Bench (100 dynamic tasks). The AOI delivers a significant +21 pp gain (p=3.0×10^{-6}), within the +17–+48 pp band reported in Table 2 for the other five models. Grok-4's absolute scores sit near the bottom of the range; manual log inspection shows this is driven by per-inference latency (∼80–180 s/call), which causes the agent to time out before completing more than a few steps on Medium and Hard tasks.

Mode	**Pass / 100**	**95% Wilson CI**	**Δ vs. Standard**	p **(McNemar)**
Standard	4	—	—	—
AOI full	25	—	+21	3.0e-06

Nemar quantifies whether two configurations differ *given a fixed run*; the seed sweep quantifies how much a single run's score moves under fresh decoding noise.

6.4 Grok-4: A Late-Arriving Vision-Capable Model

The xAI Grok-4 series [xAI, 2026] was released after the initial Table 2 runs. We wire it through xAI's OpenAI-compatible vision endpoint (`api.x.ai/v1`) and run a full standard + AOI full eval on the 100 dynamic tasks (Table 4).

Grok-4's standard score (4/100) is the lowest of any model evaluated. Manual inspection shows it is not an action-grammar problem (the agent emits valid `wait`/`click`/`fill` actions) but a latency one: a single Grok-4 call routinely takes 80–180 s, so within the per-task wall-clock budget (§5) the agent completes only one or two steps. The +21 pp AOI gain therefore lower-bounds the true effect: the AOI succeeds on tasks where a few well-placed observations suffice (e.g. one call that captures the spoken answer), but cannot unblock latency-dominated tasks that require many sequential actions. Grok-4 is a case where inference latency, not perception, becomes the binding constraint once perception is unblocked.

6.5 Newer Models: Gemini 3 Flash and Grok-4.3 / Grok-4-fast-reasoning

Do the AOI's gains hold up on the most recent model releases? We ran three additional 100-task evaluations with the same harness: (i) **Gemini 3 Flash** [Google DeepMind, 2026], a sidebar to the Gemini 2.5 Flash result; (ii) **Grok-4.3**, the latest `grok-4` series model on xAI's API; (iii) **Grok-4-fast-reasoning**, a lower-latency variant included to isolate Grok-4's perception bottleneck from its inference-latency floor (Section 6.4). Table 5 summarises the results.

Gemini 3 Flash: the gap shrinks, and not how we first read it. Gemini 3 Flash raises the standard score from 21 (Gemini 2.5) to 36 (+15 pp) and reduces the AOI Δ from +48 to +9 pp

Table 5: Additional runs on later-released models (100 dynamic tasks each). The Gemini 3 Flash sidebar checks whether the Gemini 2.5 +48 pp gain replicates on the next-generation Gemini. Grok-4.3 is the latest Grok release; Grok-4-fast-reasoning is a lower-latency variant in the same family, included to control for the inference-latency floor identified in Section 6.4.

Model	**Standard**	**AOI full**	**Δ (pp)**	p **(McNemar)**
Gemini 3 Flash	36	45	+9	0.18
Grok-4.3	25	65	+40	8.2e-10
Grok-4-fast-reasoning	19	47	+28	4.9e-06

Table 6: Decomposing the Gemini 3 Flash +9 pp AOI residual into prompt-format and perception components, using the same `standard_structured` protocol as Table 17. The Gemini 2.5 row is reproduced from Section 7.5.

Model	**Standard**	**std_structured**	**AOI full**	Δ_{prompt}	Δ_{percept}
Gemini 2.5 Flash	21	**46**	69	**+25**	**+23**
Gemini 3 Flash	36	**29**	45	**−7**	**+16**

($p = 0.18$). The natural first read—that Gemini 3 has *internalised* between-step perception—is ruled out by a four-way decomposition in Section 6.6: the +9 residual is the sum of audio +12, scaffold +9, and keyframes −12. Audio gain is structurally comparable to every other model's (+16 pp on A/B/G/H matches Gemini 2.5's +24; Claude +27; GPT-5.4 +8). The novel finding is that the keyframe-image stream—uniformly positive on every prior model generation—flips to net *negative* on Gemini 3. Dropping keyframes (mode `aoi_audio`) reaches 57/100, +12 pp above `aoi_full`: a component has crossed from net-positive to net-negative across model generations, and the observation interface is no longer a fixed bundle.

Grok-4.3 vs. Grok-4-fast-reasoning: latency is real but not the whole story. The original Grok-4 sat at standard 4, AOI full 25 (Section 6.4 attributed the low ceiling to its ∼80–180 s/call inference latency). The two follow-on runs decompose that claim. Grok-4-fast-reasoning (same base model, lower latency) reaches AOI full 47, +22 pp over the original; Grok-4.3 reaches AOI full 65, a further +18 pp. So the gap to a "saturated" Grok ceiling decomposes into +22 pp from removing latency and +18 pp from base-model improvement: the latency-bound framing in Section 6.4 was directionally right and is now quantitatively pinned.

6.6 Why Did the Gemini 3 Gap Shrink? (Scaffold-Policy Mismatch, not Internalised Perception)

We initially read the small Gemini 3 residual as the model having *internalised* the AOI's structured-record presentation. Three orthogonal measurements rule that out and replace it with a sharper reading: Gemini 3's net residual is the sum of (i) a perception gain on the same content every other model benefits from, and (ii) a Gemini-3-unique action-policy regression triggered by the `[PAGE ELEMENTS]` scaffold on transient-UI categories.

Measurement 1: per-category AOI Δ. The aggregate +9 pp number hides a stark within-model split. On the four audio categories where every CU model is fundamentally deaf (A podcast, B meeting, G phone, H interview), Gemini 3 gains $+6/+4/+5/+1 = +16$ pp—structurally identical to Gemini 2.5's $+8/+6/+8/+2 = +24$ pp, Claude's $+9/+8/+8/+2 = +27$ pp, and GPT-5.4's $+3/+1/+3/+1 = +8$ pp. On the four visual-temporal categories (C video, D carousel, I collab,

J games), Gemini 3 gains $+2/+1/+0/+0 = +3$ pp—comparable to other models' +2 to +12. The entire shortfall vs. the +48 Gemini 2.5 gain is on two categories: **E live dashboards** and **F transient UI**, where Gemini 3 alone *regresses* by −7 and −3 pp. Every other model we tested gains positively on these same two categories (Claude $+0/+5$, GPT-5.4 $+3/+6$, Gemini 2.5 $+8/+9$). Whatever shrinks Gemini 3's net residual lives entirely in this E+F regression, not in any global internalisation.

Measurement 2: action-grammar shift is universal; premature termination is Gemini-3-specific. The system prompt offers two ways to interact with a form input: visual `type("text")` (auto-targets the focused element) and DOM-selector `fill(#id, "text")` (used when `[PAGE ELEMENTS]` provides the id). Adding the `[PAGE ELEMENTS]` block shifts every cloud model toward `fill()` at similar rates: Claude 3→67/100 tasks, GPT-5.4 0→48, Gemini 2.5 3→66, Gemini 3 4→61. The grammar shift itself is therefore not the regression mechanism. What *is* unique to Gemini 3 is what happens *after* the fill: the rate of trajectories ending with a bare `done` action falls for every other model when given the scaffold (Claude 9→4, GPT-5.4 16→9, Gemini 2.5 14→9) but *rises* for Gemini 3 (8→12). Inspection of the failing `result_val=not_submitted` traces (8/100 in AOI full vs. 4/100 in standard) shows the same pattern: Gemini 3 fills the input via the listed selector, then emits `done` without explicitly clicking the submit button or pressing Enter.

Measurement 3: a four-way decomposition isolates each AOI component's per-model contribution. To distinguish "audio is the only missing content" from "Gemini 3 is internally saturated," and to separate the `[PAGE ELEMENTS]` scaffold effect from the keyframe-image effect, we run two additional controls on the full 100-task suite: (a) `standard_audio`: audio transcripts injected, but the `[PAGE ELEMENTS]` block suppressed (so action grammar and termination policy stay in the bare-`standard` regime); (b) `aoi_audio`: audio plus the full structured scaffold, but *no* keyframe images. With `standard` (36), `standard_structured` (29), and `aoi_full` (45), these form a four-way decomposition (Table 7) that inverts our prior reading and yields three sign-separated component effects on Gemini 3:

- *Audio:* $+12\,pp$ (`standard` 36 → `standard_audio` 48). Of this, +10 pp lives on the four audio categories alone, comparable in magnitude to the +16 pp `aoi_full` gets on the same categories. Gemini 3 is not audio-saturated.
- *Scaffold (`[PAGE ELEMENTS]` + structured trajectory wrapping):* $+9\,pp$ (`standard_audio` 48 → `aoi_audio` 57). Per-category the scaffold is *bi-directional* on Gemini 3: it adds +15 pp on the audio-Q&A categories where the listed input id resolves the targeting question (A: 3 → 9, B: 5 → 9, G: 4 → 8, H: 8 → 9), but it *subtracts* 4 pp on the E live-dashboard plus F transient-UI tasks (E: 6 → 1, F: 1 → 2) for the `fill`→`done`-without-submit reason traced in Measurement 2.
- *Keyframes:* $-12\,pp$ (`aoi_audio` 57 → `aoi_full` 45). This was *not* predicted by the prior-generation framing, which treated extra keyframes as monotonically additive perception. On Gemini 3 the keyframe image stream actively degrades performance: per-category, removing keyframes adds +3 pp each to A, B, and C (continuous-content tasks where the keyframes are near-duplicates of the current screenshot), neutrally affects E and F (both already at floor due to the scaffold cost), and removes only −2 pp on D (the carousel category where keyframes do carry novel visual information).

The net $(+12) + (+9) + (-12) = +9$ pp matches the `aoi_full` residual, but reframes what the small net means: Gemini 3 has substantial slack the AOI can recover (∼+21 pp with the right component mix, reaching 57/100 from 36); the default bundle is just mis-tuned for this model. One component (keyframes) crosses from net-positive on Gemini 2.5-era models to net-negative on Gemini 3, while another (structured scaffold) flips from uniformly positive to task-dependent.

Diagnostic map (revised). The observation interface is not a monolith, and its components flip sign between model generations.

Table 7: Gemini 3 Flash: four-way decomposition of the AOI residual. `aoi_audio` = audio enabled + page-elements scaffold + trajectory wrapping (no keyframes). `standard_audio` = audio enabled + standard prompt (no `[PAGE ELEMENTS]`, no trajectory wrapping). If the small AOI residual were a sign of internal saturation, `standard_audio` should be no better than `aoi_full`. Per-category numbers shown for the two categories where Gemini 3 regresses (E, F) and the four where audio is the irreplaceable signal (A, B, G, H).

Mode	Total	A+B+G+H	E+F	[PAGE_EL]?	Audio?	Keyframes?
standard	36	10	11	no	no	no
standard_structured	29	9	2	yes	no	no
standard_audio	48	20	7	**no**	**yes**	no
aoi_audio	**57**	35	3	yes	yes	no
aoi_full	45	26	1	yes	yes	yes

- *Gemini 3 Flash:* not perception- or audio-saturated. Net +9 is a three-way cancellation (+12 audio, +9 scaffold with bi-directional per-category effects, −12 keyframes). Audio+scaffold-only (`aoi_audio`) reaches 57/100—a +21 pp recovery, +12 above `aoi_full`. Components must be tuned per-model rather than bundled.
- *Gemini 2.5 Flash:* +25, +23. Both prompt-format and perception useful; neither internalised.
- *Claude Sonnet 4.6:* +19, +25. Both useful; perception the larger lift.
- *Fara-7B:* +4, +13. Reasoning-capacity-bottlenecked; the model cannot leverage richer prompt presentation.
- *Grok-4 (original):* latency-bottlenecked. The AOI full ceiling of 25 decomposes (via Grok-4-fast vs. Grok-4.3) into ∼+22 from removing latency and ∼+18 from base-model improvement.

The AOI's residual gain still functions as a diagnostic map of where a model has slack, but the dimensions are richer than "perception only": perceptual, prompt-handling, latency, reasoning-capacity, *and* action-policy compatibility with the scaffold. Calibrating the scaffold to the target model's native action grammar is the natural next step.

6.7 Per-Category Analysis

Table 8 and Figure 5 reveal category-level patterns:

Audio categories (A, B, G) show the largest gains. Standard baselines score 0–2/10 on podcast, meeting, and phone tasks across the five tabulated models—these tasks are impossible without audio perception. The AOI unlocks 3–10/10 depending on the model, with Claude achieving near-perfect scores (9/10 or 10/10).

Visual-temporal categories (C, D, F) benefit consistently. Video, carousel, and transient UI tasks improve by 1–9 points across models, with a single small regression on Fara-7B's F (4 → 3). Category E (dashboard) shows smaller gains because dashboard tasks often have sufficient information in single screenshots—the dynamic updates are periodic and the current value is always visible.

Category J (games) is an exception. Browser games require not just perception but fast, precise motor actions and strategic planning. The AOI provides better perception but does not accelerate the agent's action latency, which is the binding constraint for real-time games. Claude's score drops from 5 to 2 with the AOI. Category J has the highest step count (118 steps, 11.8 per task) and high keyframe density (0.55 KF/step; Table 14), resulting in substantial observation overhead: CLIP encoding and keyframe processing add ∼2–3 s per step on top of the model's ∼3 s inference latency, roughly doubling the agent's effective reaction time. For time-sensitive game tasks where

Table 8: Per-category task success (pass/10) for standard vs. AOI full across the five models with full per-category breakdowns; Grok-4 is excluded here for legibility because its absolute scores are latency-limited (see Section 6.4, Table 4). Categories are grouped by primary perception axis: A = audio, V = visual-temporal, I = real-time interaction.

		Claude 4.6		**GPT-5.4**		**Gemini 2.5**		**EvoCUA-32B**		**Fara-7B**	
Cat.	**Axes**	Std	AOI	Std	AOI	Std	AOI	Std	AOI	Std	AOI
A (Podcast)	A	0	**9**	0	3	0	**8**	0	**8**	0	3
B (Meeting)	A+V+I	2	**10**	2	3	2	**8**	1	6	1	5
C (Video)	V	4	**9**	5	**8**	5	6	3	6	3	5
D (Carousel)	V	4	**9**	6	7	4	7	3	4	1	3
E (Dashboard)	V	8	8	6	**9**	1	**9**	2	4	2	2
F (Transient)	V	4	**9**	2	**8**	0	**9**	0	5	4	3
G (Phone)	A+I	1	**9**	1	4	0	**8**	0	**8**	1	4
H (Interview)	A+I	7	**9**	6	7	5	7	6	**8**	1	3
I (Collab)	V+I	3	**8**	4	5	2	6	2	5	2	4
J (Games)	V+I	5	2	5	3	2	1	1	1	2	2
Total		38	**82**	37	**57**	21	**69**	18	**55**	17	**34**

Table 9: Task success by difficulty level (standard → AOI full). Easy: 30 tasks (3 per category). Medium: 40 tasks (4 per category). Hard: 30 tasks (3 per category).

	Standard			**AOI Full**		
Model	Easy	Med	Hard	Easy	Med	Hard
Claude 4.6	17/30	15/40	6/30	**28**/30	**34**/40	**20**/30
GPT-5.4	15/30	17/40	5/30	21/30	26/40	10/30
Gemini 2.5	10/30	9/40	2/30	24/30	33/40	12/30
EvoCUA-32B	6/30	8/40	4/30	20/30	26/40	9/30
Fara-7B	9/30	5/40	3/30	19/30	11/40	4/30

actions must occur within specific windows, this added latency causes the agent to miss deadlines it would otherwise meet. This confirms that the AOI is designed for human-paced computer use, not real-time interactive applications.

EvoCUA-32B shows disproportionate gains on audio tasks. The model jumps from 0/10 to 8/10 on both podcast (A) and phone (G) categories, suggesting that audio perception particularly benefits models whose visual grounding is weaker.

6.8 Difficulty Breakdown

Table 9 shows that the AOI lifts all difficulty levels. For Claude, easy task success rises from 57% to 93%, medium from 38% to 85%, and hard from 20% to 67%. The pattern is consistent across models: the AOI unlocks perception-gated tasks at every difficulty, but hard tasks remain challenging because they require both perception *and* sophisticated reasoning. Fara-7B's small gain on medium/hard (5→11, 3→4) compared to its large gain on easy (9→19) suggests that reasoning capacity, not perception, becomes the bottleneck for the 7B model on harder tasks.

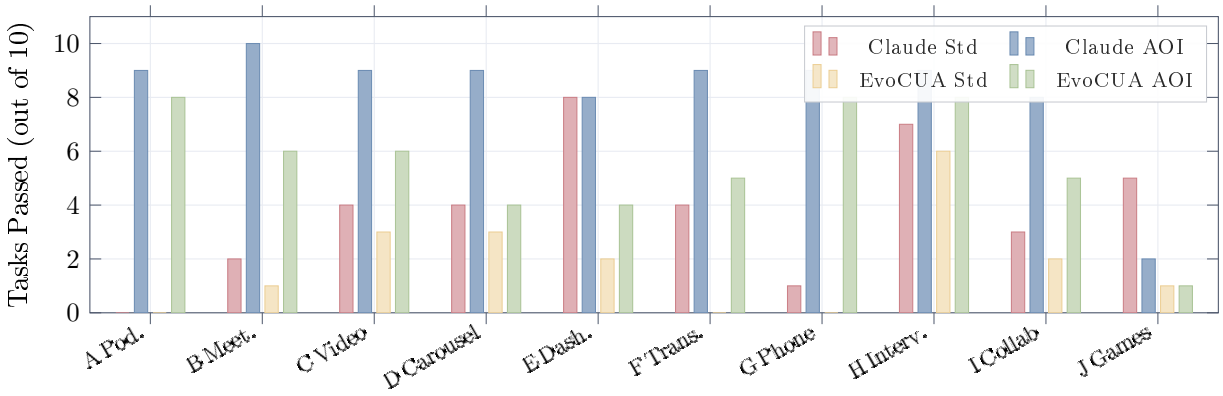

Figure 5: **Per-category results for Claude Sonnet 4.6 and EvoCUA-32B.** Audio-dependent categories (A, B, G) show the largest gains—from near-zero to 6–10/10. Visual-temporal categories (C, D, F) improve consistently. Category J (games) is the exception: real-time action latency, not perception, is the bottleneck. Category E (dashboard) shows minimal change for Claude because current values are visible in single screenshots.

Table 10: Ablation study on Claude Sonnet 4.6: contribution of each AOI component. All configurations include per-category breakdown for categories most affected.

Mode	**Total** /100	**A**	**B**	**C**	**G**	**H**
		(pass/10, selected categories)				
Standard	38	0	2	4	1	7
Uniform 1 FPS	58	0	4	**10**	1	8
Uniform 3 FPS	56	2	5	8	1	7
Pixel-diff	58	1	5	9	1	8
Random keyframes	56	1	6	8	1	8
AOI visual only	58	1	5	9	1	8
AOI visual + ASR	64	1	4	**10**	4	**9**
AOI full	**82**	**9**	**10**	9	**9**	**9**

6.9 Ablation: Observation Mode Analysis

Table 10 and Figure 6 reveal the contribution of each AOI component (the complete per-category breakdown for all eight ablation modes is in Appendix A, Table 20):

Selection method does not affect accuracy (56–58%). We evaluated five visual-only keyframe strategies: (a) *Uniform 1 FPS*: capture at a fixed rate; (b) *Uniform 3 FPS*: higher fixed rate; (c) *Pixel-diff*: capture when >1% of pixels change; (d) *Random*: randomly sample the same number of frames as adaptive methods; (e) *CLIP-based adaptive*: two-stage filter (pixel gate → CLIP-ViT-B/16 cosine distance exceeding threshold θ; Algorithm 1). All five converge to a narrow performance band of 56–58%, and pairwise McNemar tests on the task-level outcomes find no significant differences between any pair (every $p > 0.5$; see Table 12). The +18–20 pp gain over the single-screenshot baseline ($p = 8.8 \times 10^{-5}$ for Standard → Uniform 1 FPS) comes from breaking the agent's blindness between steps, regardless of whether frames are chosen uniformly, randomly, or adaptively. This result simplifies the practical recommendation: practitioners can use whatever keyframe strategy fits their latency and compute budget.

Open-source replication on Qwen3-VL-32B. We replicate this convergence on an open-

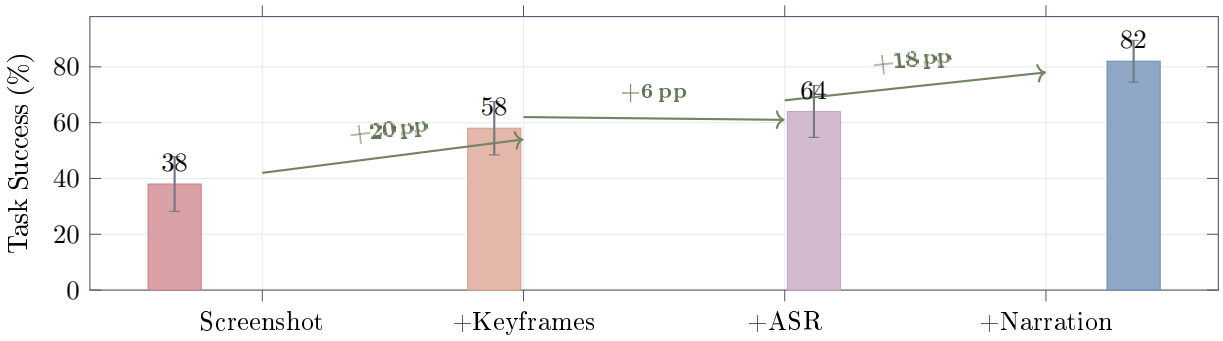

Figure 6: **Ablation: progressive contribution of each AOI component** (Claude Sonnet 4.6). Inter-step keyframes provide +20 pp (any selection method achieves this; see Table 10). ASR adds +6 pp (hearing speech and sounds). Visual narration adds +18 pp—the largest single-component gain—of which a significant +8 pp is from *persistent text memory* ($p = 0.039$) and the remaining +10 pp is a directional CoT effect at the inference time of narration generation that does not reach significance at N=100 (Section 7.2). Error bars are approximate 95% intervals using a symmetric Gaussian span (exact Wilson intervals in Table 2).

Table 11: Selection-method ablation on an open-source vision model (Qwen3-VL-32B-Instruct via OpenRouter), 50 visual-temporal tasks (categories C, D, E, F, J). The three selection methods cluster within a narrow band, mirroring the Claude Sonnet 4.6 pattern. Pairwise McNemar tests find no significant differences ($p > 0.4$ for every pair), confirming that the convergence finding is not Claude-specific.

Selection method	**Pass / 50**	**Vs. adjacent**
Uniform 1 FPS	26	—
Pixel-diff	27	vs. Uniform: p =0.69
Random keyframes	27	vs. Pixel-diff: p =1.00

source vision model to rule out that it is a Claude-specific phenomenon. EvoCUA-32B and Fara-7B could not be re-run because the host's vLLM cannot start under the current NVML driver state (Section 9); we substitute Qwen3-VL-32B-Instruct [Qwen Team, 2025], a similarly-sized open-source vision-language model accessed through OpenRouter. We evaluate the three cheapest-to-run selection methods (Uniform 1 FPS, Pixel-diff, Random) on the 50 visual-temporal tasks (categories C, D, E, F, J)—the subset where keyframe selection actually differs. Table 11 reports the result. The three methods cluster within a narrow band on the open-source model as well, with no pairwise McNemar significance ($p > 0.4$ for every pair), exactly mirroring the Claude pattern. This is direct evidence that the convergence is mechanism-driven (any inter-step frame breaks the agent's blindness regardless of how it is selected), not a quirk of a single model.

Adding ASR (+6 pp). Whisper speech transcription lifts performance from 58% to 64%. The gain concentrates on audio-dependent categories (G: 1→4, H: 8→9). Categories A and B show minimal improvement because podcast and meeting comprehension require understanding the *content* of speech over multiple steps, not just detecting that speech occurred.

Full AOI (+18 pp over visual+ASR; $p = 5.3 \times 10^{-4}$). The full AOI with visual narration reaches 82%. The large gap from 64% to 82% demonstrates that visual narration—converting ephemeral keyframe images into persistent text—is critical for tasks requiring recall of earlier observations. The improvement is especially pronounced on podcast (A: 1→9) and meeting (B: 4→10)

Table 12: Pairwise McNemar's exact mid-p tests between successive ablation tiers on the same 100 tasks (Claude Sonnet 4.6). Each row tests whether the labelled configuration differs significantly from the previous one. "b" / "c" are the discordant counts (success in only the previous / only the current configuration). The selection-method group is statistically indistinguishable; the between-tier transitions are highly significant.

Comparison	**Total**	b	c	p
Standard → Uniform 1 FPS	38 / 58	3	23	8.8×10^{-5}
Uniform 1 FPS → Uniform 3 FPS	58 / 56	7	5	0.77
Uniform 3 FPS → Pixel-diff	56 / 58	4	6	0.75
Pixel-diff → Random keyframes	58 / 56	4	2	0.69
Random keyframes → CLIP adaptive	56 / 58	4	6	0.75
CLIP adaptive → AOI visual+ASR	58 / 64	4	10	0.18
AOI visual+ASR → AOI full	64 / 82	4	22	5.3×10^{-4}

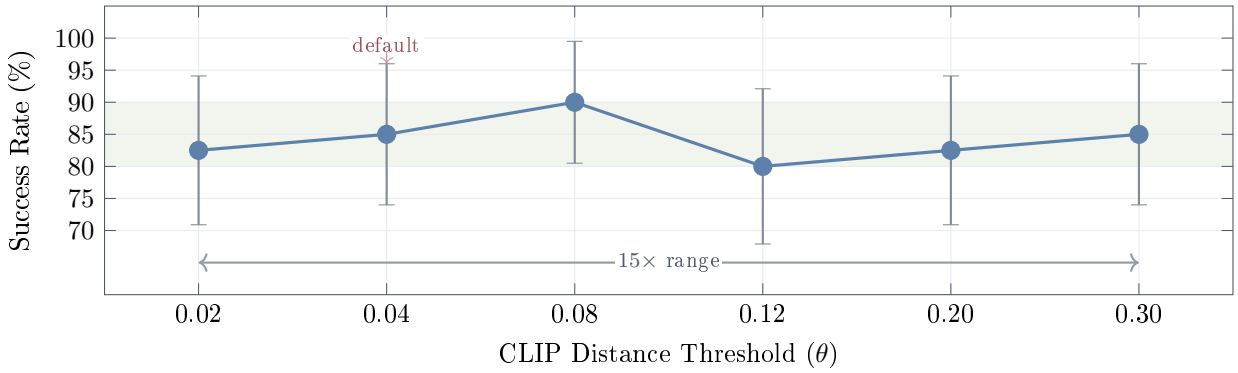

Figure 7: **CLIP threshold (θ) sensitivity** on 40 visual tasks (categories C–F) with Claude Sonnet 4.6. Error bars are 95% Wilson confidence intervals at N=40 (±9.5–12.1 pp). Point estimates remain within an 80–90% band (shaded) across a 15× range of θ, but the CIs overlap heavily, so we cannot distinguish individual thresholds at this sample size. The bimodal distribution of CLIP distances—near-zero for unchanged content, large for meaningful changes—makes the method naturally insensitive to θ.

tasks, where narration enables the agent to accumulate information across many steps.

Resolving the CoT-vs-memory confound. The +18 pp gain could in principle conflate two effects: (a) the *persistent text memory* that survives keyframe pruning, and (b) a *chain-of-thought* effect from generating the narration itself, which forces the model to explicitly reason about visual content before selecting an action. We disentangle these with the *narration-discarded* ablation: the model still produces a narration in every inference call (so any CoT-style benefit at inference time still applies), but we drop the narration from the trajectory before persisting the step. Section 7.2 reports the result.

7 Analysis

7.1 CLIP Threshold Sensitivity

Figure 7 shows the CLIP threshold θ sweep across a 15× range from 0.02 to 0.30, evaluated on the 40 visual tasks (categories C–F) where keyframe selection is relevant. Point estimates remain

Table 13: Narration ablation on Claude Sonnet 4.6 (100 dynamic tasks). McNemar's exact mid-p tests on the discordant tasks. The narration-discarded configuration sits between the two endpoints, indicating that both the inference-time chain-of-thought effect (+10 pp from ASR to ND) and the persistent text-memory effect (+8 pp from ND to full) contribute to the full +18 pp gain. Only the persistent-memory effect reaches significance at N=100. Per-category breakdown shows the persistent-memory effect concentrates on audio + temporal-visual tasks (F: 4→9, E: 7→8) where state from prior steps must be retained as text once the source images are pruned.

Configuration	**Total**	A	B	C	D	E	F	G	H	I	J	**Vs. adjacent**
AOI visual+ASR (no narration)	64	1	4	10	9	9	9	4	9	6	3	—
AOI full *narration discarded*	74	9	10	8	8	7	4	9	9	7	3	vs. ASR: $p = 0.12$
AOI full (narration as persistent memory)	**82**	9	10	9	9	8	**9**	9	9	8	2	vs. disc.: $p = 0.039$

within an 80–90% band throughout, demonstrating that for practitioners who choose the CLIP-based variant, threshold selection is not critical. There is no cliff: even at $\theta = 0.30$ (very aggressive filtering that captures only major visual changes), the point estimate remains at 85%. At $\theta = 0.02$ (very sensitive, capturing small changes), it is 82.5%. The point estimate peaks at $\theta = 0.08$ (90%), but with N=40 the 95% Wilson confidence intervals on every point span ∼10–12 pp and overlap heavily—we therefore do not claim a true sweet spot, only that the method is robust across a 15× range of θ and that any reasonable choice will work.

7.2 Narration: Persistent Memory or Chain-of-Thought?

The full AOI's largest single gain comes from visual narration (+18 pp). This component does two things at once: it (i) asks the CU model to articulate the new visual content as a side-output during inference, and (ii) persists that text in the trajectory so it survives keyframe pruning. To separate these effects we run the *narration-discarded* ablation: the model is prompted exactly as in AOI full and produces a narration each step (so any inference-time chain-of-thought benefit is preserved), but the narration string is dropped from the trajectory before it can be reused as context. The keyframe and audio pipelines are otherwise identical to AOI full.

Table 13 reports the result. The narration-discarded score is 74/100, sitting between the visual+ASR baseline (64) and the full AOI (82). The +18 pp gap therefore decomposes into: (i) a +8 pp contribution from persistent text memory of past visual observations (74 → 82, McNemar $p = 0.039$), which is the *only significant component at* N=100; and (ii) a directional +10 pp from inference-time chain-of-thought during narration generation (64 → 74, $p = 0.12$), which is consistent in sign and magnitude but underpowered—reaching $p < 0.05$ at this effect size would require $N \sim 300$. The well-supported claim is that persistent text memory is, on its own, a load-bearing component of the AOI; the CoT contribution is a plausible secondary effect whose confirmation we leave to a larger evaluation.

7.3 Per-Step Observation Statistics

Table 14 and Figure 8 show the per-step observation statistics for the AOI. The data confirms the gate-suppression design: audio-only categories (G, H) produce zero keyframes, while visual-only categories (D, E, F, J) produce zero audio activations. Category D (carousel) has the highest keyframe density at 1.38 per step because carousel animations produce frequent visual changes. Category A (podcast) has the lowest keyframe rate (0.09) because podcast tasks involve a mostly

Table 14: Per-step observation statistics for **Claude Sonnet 4.6 + AOI full**, broken down by task category. %KF = percentage of steps where at least one keyframe was captured. KF/Step = average keyframes per step. %Audio = percentage of steps with audio transcription. Audio-heavy categories (A, B, G, H) show 26–74% audio activation. Visually dynamic categories (D, J) average 0.55–1.38 keyframes per step. Audio-only categories (G, H) produce zero keyframes—the pixel and CLIP gates correctly suppress extraction when only audio changes.

Cat.	Domain	Steps	%KF	Tot. KF	KF/Step	%Audio
A	Podcast	35	8.6	3	0.09	42.9
B	Meeting	34	50.0	18	0.53	73.5
C	Video	39	28.2	12	0.31	2.6
D	Carousel	61	60.7	84	1.38	0.0
E	Dashboard	52	28.8	18	0.35	0.0
F	Transient	38	15.8	7	0.18	0.0
G	Phone	38	0.0	0	0.00	26.3
H	Interview	27	0.0	0	0.00	37.0
I	Collab	39	10.3	4	0.10	17.9
J	Games	118	36.4	65	0.55	0.0
All		**481**	**28.3**	**211**	**0.44**	**14.1**

static UI with audio content. Overall, only 28.3% of steps produce keyframes and 14.1% activate audio processing—the gates are effective at suppressing unnecessary computation.

7.4 Efficiency

Table 15 reveals two distinct patterns—one for token cost, one for wall-clock latency. *Token cost falls for every cloud model*: Claude \$2.72 → \$1.35 (−50%), GPT-5.4 \$3.23 → \$2.76 (−15%), Gemini 2.5 Flash \$0.32 → \$0.16 (−50%). This is because better perception leads to fewer steps per task: Claude uses 4.8 steps with AOI vs. 10.7 without, Gemini drops from 11.5 to 5.4, and EvoCUA-32B from 9.3 to 5.3. The observation overhead (12–32 s per task) is more than offset by saving 2–6 model calls per task.

Wall-clock latency, however, does not always fall. For four of five models, total task time *rises* with the AOI: Claude 40.6 s → 46.2 s (+14%), GPT-5.4 26.6 s → 50.6 s (+90%), Gemini 55.3 s → 61.9 s (+12%), and Fara-7B 35.1 s → 90.7 s (+158%). Only EvoCUA-32B sees absolute time *decrease* (117.0 s → 113.9 s, −3%) because its very high per-step model latency (∼10 s/step) means step reduction dominates the added observation overhead. Summary: the AOI is consistently cheaper in dollars, but for fast-inference models it is slower in wall-clock time because per-step observation work exceeds the latency saved per skipped step. For batch automation or cost-sensitive production the AOI is unambiguously preferred; for latency-sensitive interactive use, the trade-off is per-model.

7.5 Static-Task Verification (Static-50)

A key design claim is that the AOI does not degrade performance on purely static, silent desktop work. We verify this on **Static-50**, a 50-task baseline (22 easy, 16 medium, 12 hard) of pure HTML tasks: form filling, table reading, calculation, sequence puzzles, and policy reasoning. None of the pages contain animations, timers, video, or audio; the rendered DOM is identical at $t = 0$ and $t = \infty$. This is large enough to detect a 5–10 pp difference between modes with reasonable power;

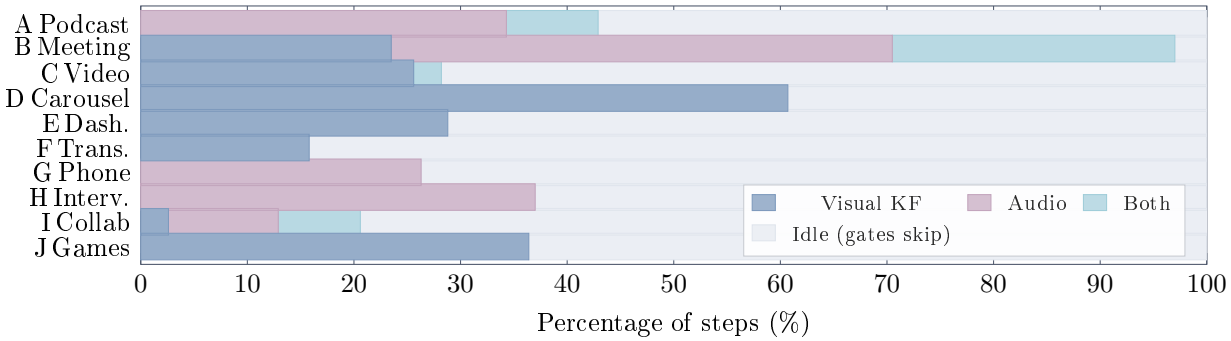

Figure 8: **Observation activity per category.** Stacked bars show the percentage of steps in which each observation channel fires. The gray "idle" portion is steps where both gates suppress processing. Categories cluster: D is highly visual; G and H are audio-only; B uses both. Weighted by step count, ∼61% of all steps are idle.

Table 16 reports the result.

Across all 50 AOI-mode runs on Static-50, the pixel gate captured only 7 keyframes total (an average of 0.09 per step, all triggered by user-driven UI feedback such as form-field focus or checkbox state changes) and the volume gate produced zero audio activations. The AOI scores 50/50 vs. Standard's 43/50. The 7 Standard failures are all multi-element form tasks (S-E3 dropdown selection, S-E4 contact form, S-E9 reference-code copy, S-M2 address form, S-M8 settings toggles, S-M9 meeting schedule, S-H5 email reply)— i.e., they require the agent to act on multiple form fields in one trajectory, exactly the workload where the structured observation record's DOM element list and prior-step text trajectory help most. The AOI's perfect 50/50 on a workload with near-zero perception extraction (7 keyframes from incidental UI feedback, 0 audio segments across ∼80 steps) is therefore consistent with the prompt-format explanation, not with a hidden perception gain.

Structured-prompt confound: a direct measurement on DynaCU-Bench. Because the gates suppress almost all extraction on Static-50 (7 keyframes, 0 audio segments across 50 tasks), the +14 pp advantage there is *not a perception gain*: it is an effect of the AOI's structured observation-record prompt format itself (DOM element list, prior-step trajectory summary, explicit context/new sectioning) over a raw screenshot. We measure this directly on DynaCU-Bench via a `standard_structured` mode that supplies the AOI's prompt scaffolding while *disabling all keyframe and audio extraction*: its gain over Standard is the prompt-format effect, and AOI full minus standard_structured is the perception-only effect.

Table 17's five models (three cloud + two open-source) directly test the prompt-format effect on DynaCU-Bench; Gemini 3 Flash is reported separately in Table 6 (Section 6.6). The open-source rows required patching vLLM's CUDA platform detection to bypass a broken NVML library (Section 9). Both Δ_{prompt} and Δ_{percept} are positive for every model: the structured prompt format and the perception channel each contribute independently rather than one re-explaining the other. The split varies with model—Claude Sonnet 4.6 is perception-leaning (+25 vs. +19 pp), Gemini 2.5 Flash and GPT-5.4 are roughly balanced (+23 vs. +25 and +9 vs. +11 pp), Fara-7B is sharply perception-leaning (+13 vs. +4 pp), consistent with the smallest model being reasoning-capacity-bottlenecked and unable to leverage a richer prompt format. Across all five models the worst-case framing that the entire AOI gain is a prompt-format artifact is decisively ruled out; the most favourable (perception is the only mechanism) is also too strong. For Gemini 2.5—which scored only 21/100 with raw screenshots—the prompt-format component alone recovers +25 pp: for video-native models, part of what the AOI buys is not new visual information but a better way to present the existing visual +

Table 15: Efficiency and cost across observation configurations on the 100-task benchmark for the five models with full instrumented runs. Grok-4 is excluded because its ∼80–180 s/call latency floor distorts the cost/latency picture (Section 6.4). Avg Steps = mean steps per task (lower is better). Obs Overhead = total observation processing time per task, including screenshot capture (∼2 s/step) plus CLIP encoding, keyframe selection, and audio processing for AOI configurations. Tokens are accounted as input + output per task; per-step prompt tokens are taken from each provider's reported usage when available, with a fallback estimate of $\lceil chars/4 \rceil$ for text and $\lceil wh/(14^2) \rceil$ for each image at vendor patch size 14 (1024×768 image → ∼4,000 tokens, consistent with vendor docs); see `experiments/compute_tokens.py`. $ / 100 tasks computed at May 2026 list prices: Claude Sonnet 4.6 $3/Mtok in, $15/Mtok out; GPT-5.4 $5/$15; Gemini 2.5 Flash $0.30/$2.50; EvoCUA-32B and Fara-7B run locally. Despite per-step observation overhead, the AOI reduces *cost* by 15–50% across the three cloud models (50% on Claude, 15% on GPT-5.4, 50% on Gemini 2.5 Flash) because the step count drops more than the per-step input grows.

Configuration	Steps	Time (s)	Lat. (s)	Obs (s)	Tokens (k/task)	$/100 tasks
Claude Sonnet 4.6						
Standard	10.7	40.6	26.3	2.5	7.6	$2.72
AOI full	**4.8**	46.2	15.2	12.0	**3.8**	**$1.35**
GPT-5.4						
Standard	10.0	26.6	14.4	2.4	5.8	$3.23
AOI full	**7.6**	50.6	11.2	17.8	**5.0**	**$2.76**
Gemini 2.5 Flash						
Standard	11.5	55.3	40.4	2.6	7.6	$0.32
AOI full	**5.4**	61.9	20.6	20.6	**4.0**	**$0.16**
EvoCUA-32B (local)						
Standard	9.3	117.0	99.0	2.3	5.8	—
AOI full	**5.3**	113.9	72.0	22.6	**4.0**	—
Fara-7B (local)						
Standard	13.4	35.1	24.0	2.5	9.2	—
AOI full	**9.3**	90.7	17.0	31.6	**7.1**	—

textual context.

7.6 Comparison with Streaming Multimodal Baselines

A natural alternative to the AOI is a *streaming, end-to-end multimodal model* that bundles perception with reasoning into a single trained model. The two production endpoints exposing this paradigm today are Gemini Live [Google, 2025] (audio + video streams + tool calls) and the OpenAI Realtime API [OpenAI, 2024] (audio + tool calls; vision via the standard chat endpoint). Neither is shipped as a CU agent: each is a voice-first assistant with function-calling. We adapt them to DynaCU-Bench by opening one streaming session per task, streaming the page audio in via PulseAudio, sending screenshots as multimodal turns, and translating tool calls into browser actions (implementation: `aoi/realtime_baselines.py`). We probe two OpenAI Realtime model identifiers at evaluation time (May 2026): `gpt-realtime` (beta tier, accepts our project key) and `gpt-realtime-2.0` (GA tier; our project does not have GA access). Neither accepts vision tokens *inside* the Realtime websocket session, so a fair vision + audio comparison requires a secondary non-realtime call for the screenshot. We use chat-completions with `gpt-4o-realtime-preview` as

Table 16: Static-50 verification: Claude Sonnet 4.6 on 50 pure-HTML tasks (no dynamic content). The AOI's pixel gate suppresses keyframe extraction (total keyframes = 0) and the volume gate suppresses audio processing (audio segments = 0): the perception pipeline runs but never produces an observation, validating the gates' suppression behaviour. The AOI's success rate is comparable to Standard, confirming no degradation on static work.

Mode	Pass/50	Avg Steps	Total KF	Audio Seg.
Standard	43 (86.0%)	3.26	0	0
AOI full	**50** (100.0%)	**1.62**	7	0

Table 17: Decomposing the AOI gain into prompt-format vs. perception on DynaCU-Bench (100 tasks). **Standard** is a raw screenshot. **standard_structured** adds the AOI's structured observation-record prompt format (DOM element list, prior-step text trajectory) but no keyframes and no audio. **AOI full** is the full perception layer. Δ_{prompt} isolates the prompt-format effect; Δ_{percept} isolates the keyframe + audio + narration contribution. The two open-source rows are included after patching the vLLM platform detection to bypass the broken NVML library (see Section 9). Gemini 3 Flash is reported separately in Table 6.

Model	Standard	std_structured	AOI full	Δ_{prompt}	Δ_{percept}
Claude Sonnet 4.6	38	**57**	82	**+19**	**+25**
GPT-5.4	37	**48**	57	**+11**	**+9**
Gemini 2.5 Flash	21	**46**	69	**+25**	**+23**
Open-source local models					
EvoCUA-32B	18	34	55	+16	+21
Fara-7B	17	**21**	34	**+4**	**+13**

the OpenAI backbone—the integration path the OpenAI Realtime cookbook recommends for "give a voice assistant eyes" and what a practitioner deploying the OpenAI streaming-voice stack today would use. The Gemini baseline uses Gemini 2.5 Flash native-audio, the highest function-calling-capable Live tier available.

Adapter sanity check. To rule out broken-adapter as an explanation for the audio-subset numbers, we evaluate both baselines on a five-task purely-visual sanity set across four categories (C-E1, E-E1, F-E1, F-E2, I-E1; two F tasks cover both transient-banner subtypes). Each adapter emits valid tool calls of every supported type (`click`, `fill`, `type_text`, `scroll`); the browser executes them; the resulting DOM is scored by the main benchmark's success criterion (Table 19; per-task detail in Appendix F). Both baselines hit action-grammar success on 5/5 tasks but task-correct success on 0/5: clicks land on the wrong UI elements, typed text is hallucinated, and the model commits before reading the page. On the cookie-banner task (F-E2), OpenAI Realtime clicks "dismiss" instead of "accept"—the click reached the DOM (`cookies_dismissed` is recorded), but the model chose the wrong target. AOI-equipped Claude Sonnet 4.6 passes 4/5 on the same sanity set, confirming the tasks are solvable through a vision-grounded CU loop. The audio-subset failures below therefore reflect intrinsic limitations of streaming-voice-with-tool-calls on grounded computer-use, not adapter scaffolding.

Both streaming baselines underperform the AOI by a wide margin on this audio-focused subset (column codes match Table 1: A podcast, B meeting, G phone, H interview). OpenAI Realtime succeeds only on the simplest interview tasks (3/3), where the user's question is short enough to answer from a single audio chunk; on podcast and meeting tasks it emits `wait()` tool calls indefinitely

Table 18: Comparison against monolithic streaming baselines on a 12-task balanced cross-category subset (3 tasks each from podcast A, meeting B, phone G, interview H). We adapt each streaming API to DynaCU-Bench by opening one session per task, streaming page audio + screenshots in, and translating returned tool calls into browser actions. Each baseline is scored on the same 12 task IDs the AOI is scored on in Table 2. OpenAI backbone and Gemini Live tier choices are detailed in the body below.

Configuration	A	B	G	H	Total / 12
OpenAI Realtime (gpt-4o-realtime-preview)‡	0	0	0	3	3 (25%)
Gemini Live (2.5 flash native-audio)	0	0	0	0	0 (0%)
AOI full (Claude Sonnet 4.6), same tasks	3	3	3	3	**12** (100%)

‡ The Realtime websocket does not accept vision tokens in the same session, so vision is supplied through chat-completions; see backbone-choice note in caption.

Table 19: Adapter sanity check for the streaming baselines. Five purely-visual tasks (no audio comprehension required). **Adapter exercised**: counts the tasks where the model emitted at least one valid tool call and the browser executed it (action-grammar success). **Task-correct**: counts the tasks where the resulting DOM passed the benchmark's success check. The adapter is fully exercised on every task; task-correct failures are content errors (wrong button, wrong text, hallucinated answer), not infrastructural.

Baseline	Adapter exercised	Task-correct (visual sanity)	Audio subset (12 tasks)
OpenAI Realtime (gpt-4o + vision tools)	5/5	0/5	3/12
Gemini Live (2.5 flash native-audio)	5/5	0/5	0/12
AOI full (Claude Sonnet 4.6) reference	5/5	4/5	12/12

without ever extracting and acting on the spoken content. Gemini Live, configured with the native-audio model and function-calling tools, scores zero on this subset. Manual inspection of trajectories shows the model returns very few tool calls per session, and when it does, it tends to issue clicks at non-actionable coordinates rather than commit to a typed answer; the model appears to be optimised for verbal-response use cases (audio in, audio out) rather than for grounded function-call decisions on a screenshot. Together with the adapter sanity check above, these failures reflect a limitation of the streaming-voice-model paradigm itself rather than adapter scaffolding. The combined picture validates the central design hypothesis: *a streaming voice model with function-calling is not a substitute for a perception layer that persists what it heard as text in the trajectory.*

7.7 Cross-Engine ASR Validation (DynaCU-Real-Local)

To verify that the AOI's audio pipeline does not silently fail under a different acoustic profile than the edge-TTS audio used in DynaCU-Bench, we re-ran 12 audio tasks with audio rendered by `espeak` (a formant synthesizer) and screencasts driven by real `asciinema` v2 cast files. Standard and AOI full both score 11/12 (the same single failure for both, and pretraining suffices for the rest; Table 21 in Appendix E), so the result is *not* a perception comparison—it is a no-degradation check: Whisper produces intelligible transcripts from the espeak audio in every case (verified by manual inspection of the transcripts). Designing real-recording tasks whose answers are not recoverable from CU-model pretraining is harder than it sounds and is left to follow-up work (Section 9). Harness details, per-task breakdown, and asset sources are in Appendix E.

8 Related Work

Computer-use agents. The CU agent landscape spans closed-source systems [Anthropic, 2024a, OpenAI, 2025, Google DeepMind, 2025, Andreux et al., 2025] and open-source models [Wang et al., 2025a,b, Awadallah et al., 2025, Ye et al., 2025, Song et al., 2025]. Agent S3 [Simular AI, 2025] demonstrates competitive open-source performance on OSWorld. A recent CU agent survey [Sager et al., 2025] catalogs many agents and datasets and identifies several open gaps, but does not identify the observation modality as one. All of these agents operate through the screenshot–reason–act loop; none addresses dynamic visual content or audio perception.

Real-time multimodal models. Gemini Live [Google, 2025] and the OpenAI Realtime API [OpenAI, 2024] process continuous audio/video/text streams in real time, bundling perception with reasoning into a single trained model. Our work is complementary: the AOI decouples perception from reasoning, so the same perception layer benefits any CU model without retraining. Section 7.6 compares the two paradigms head-to-head on the audio-focused DynaCU-Bench subset.

Video understanding for GUI. GUI-World [Chen et al., 2024], VideoWebArena [Jang et al., 2024], and CUA-Suite [Chen et al., 2025] identify the observation problem through benchmarking. MemGUI-Bench [Park et al., 2025] reveals memory gaps in GUI agents. These benchmarks identify the problem but do not propose solutions compatible with existing CU agents.

Keyframe selection. VideoTree [Wang et al., 2024] and AKS [Tang et al., 2025] use CLIP-based frame selection for offline video QA. Apollo [Zohar et al., 2024] finds that FPS sampling outperforms uniform sampling and identifies optimal token budgets. All prior work targets offline video understanding; our contribution is real-time observation filtering for CU agents, where the two-stage design (sub-millisecond pixel gate followed by $\sim$7 ms CLIP only on changed frames) keeps the per-frame cost low enough to run continuously in the background between agent steps.

Adaptive middleware for GUI agents. Cradle [Tan et al., 2024] includes frame difference analysis for games—the closest predecessor to the AOI's perception layer—but uses pixel-level differencing without semantic filtering and has no audio processing. ScreenLLM [Jin et al., 2025] captures stateful screen schemas with keyframe extraction, complementing visual narration but without audio or adaptive observation. StreamAgent [Yang et al., 2025] and Dispider [Qian et al., 2025] separate perception from response generation for streaming video, paralleling the AOI's architecture, but neither addresses CU agent observation.

Structured agent interfaces. A complementary line of work bypasses the perception problem by having websites or applications expose structured interfaces directly to agents: the Model Context Protocol [Anthropic, 2024b] for tool/data exchange and declarative web specifications such as NLWeb [Microsoft, 2025] for site-side agent endpoints. These approaches are powerful where adoption exists, but require website cooperation; the AOI enables perception on arbitrary, unmodified web content (any HTML/audio rendered to a browser, including legacy or third-party pages with no agent endpoint).

9 Limitations

Action latency is unsolved. The AOI extends perception but does not accelerate decision-making. CU models still require 1–5 seconds per inference step, making the approach unsuitable for real-time games or fast interactive applications (as evidenced by category J results).

Temporal resolution floor ($\sim$300 ms). Sampling at $\sim$3 Hz means events shorter than $\sim$300 ms can fall entirely between samples. This primarily affects category J (browser games) where targets appear briefly. For human-paced computer use (meetings, web browsing, presentations) this reso-

lution is sufficient; for high-speed visual content, information is irretrievably lost.

Context-window pressure on keyframe-heavy tasks. On categories D and E, accumulated keyframe images can exceed the CU model's context window—we observed overflow at 8,192 tokens on some local models. Retaining text narrations rather than images in long trajectories (Section 7.2) sidesteps this almost entirely.

Reasoning capacity remains a separate bottleneck. For smaller models (Fara-7B), improved perception does not help with tasks requiring multi-step reasoning, arithmetic, or complex instruction following: better inputs do not improve reasoning over those inputs. Fara-7B's gain on hard tasks is small ($3 \rightarrow 4$) compared to easy tasks ($9 \rightarrow 19$).

Statistical power. Each cell in our main table is one trial of 100 binary tasks; we use paired McNemar's exact tests to assess differences but the data is not powered for fine-grained ranking between configurations whose marginal scores are within a few percentage points.

Structured-prompt vs. perception confound (directly measured). The original concern was that the AOI's structured observation-record prompt format could explain part of the gain on its own. We measure this directly on DynaCU-Bench with a `standard_structured` mode (Table 17 in Section 7.5) for five models, with Gemini 3 Flash reported in Table 6. The two open-source rows required additional engineering: the host had an NVML/userspace mismatch (NVML 595.58 vs. kernel module 590.48.01) that prevented vLLM 0.19.0 from detecting CUDA at all—its CUDA platform plugin only calls `pynvml.nvmlInit()` and returns "no CUDA available" when it fails. We patched the plugin to fall back to `torch.cuda.is_available()` when NVML is broken, and relaxed an overly strict free-memory assertion in vLLM's worker that fired whenever another GPU user (the Whisper service) released memory during vLLM's startup profiling. With those two patches we ran Fara-7B and EvoCUA-32B in `standard_structured` mode on the same host. For Claude perception is the larger component (+25 pp vs. +19 pp prompt-format); for Fara-7B the split is even more perception-skewed (+13 pp vs. +4 pp), consistent with small models being reasoning-bottlenecked; for Gemini 2.5 Flash and GPT-5.4 the two are roughly balanced.

Narration quality is bounded by the CU model. Long-term visual memory depends on the CU model producing accurate text descriptions. Errors—a misread number, a hallucinated UI element—are permanently encoded in the trajectory.

Audio limited to speech transcription. Our implementation uses Whisper for speech-only ASR rather than a multimodal audio model. Non-speech audio events (notification sounds, alert beeps) are not transcribed. A multimodal audio model (e.g., Qwen3 Omni) could extend coverage to the full audio scene.

Benchmark scope. DynaCU-Bench tasks execute in a controlled browser environment with synthesized audio. Performance on real-world dynamic content (live video calls, real YouTube, native desktop applications) may differ.

Open-source replication on a substitute model. For the selection-method convergence finding we additionally evaluate Qwen3-VL-32B-Instruct [Qwen Team, 2025] via OpenRouter as an independent open-source vision model. The convergence pattern reproduces (Section 6.9, Table 11). The EvoCUA-32B and Fara-7B `standard_structured` rows required the vLLM platform-detection patch described above; the patched detection logic is upstream-style and is released with the code for reproducibility.

Real-content task design. The DynaCU-Real-Local harness (Appendix E) verifies cross-engine ASR robustness, but its tasks have answers recoverable from the CU model's pretraining, so they cannot also serve as a perception comparison. Designing real-recording tasks whose ground-truth answers are not in the model's pretraining is harder than it sounds and is left to future work.

10 Conclusion

Computer-use agents cannot perceive dynamic content because observation is tied to action: one screenshot per step. We argue the agent-computer *observation* interface is a separable, underexplored design axis—the natural counterpart to the action interface SWE-agent [Yang et al., 2024] surfaced—and introduce the AOI, a perception layer transparent for static tasks, adaptive for dynamic content, and compatible with any existing CU model without retraining. Six CU agents equipped with the AOI gain +17 to +48 pp on DynaCU-Bench (paired McNemar $p < 10^{-3}$ for every model); the strongest reaches 82% on Claude Sonnet 4.6 (3-seed mean 78.7%, std 3.1 pp).

Two findings reframe how we think about CU perception. First, keyframe *selection* does not significantly affect accuracy—uniform, random, pixel-difference, and CLIP-based methods all converge to ∼58% on Claude and on the open-source Qwen3-VL-32B—so the critical factor is capturing *any* inter-step frames, not how they are selected. Second, the largest single contributor is visual narration (+18 pp), of which a significant +8 pp comes from *persistent text memory of past visual observations* ($p = 0.039$) and a directional +10 pp from inference-time CoT during narration ($p = 0.12$). Observation memory, not just reasoning and action, is a load-bearing bottleneck—and one that admits a practical, training-free solution.

Three further results sharpen the picture. Token cost falls with the AOI on every cloud model (15–50%) because better perception yields fewer steps per task; wall-clock time rises for fast-inference models and falls for slow ones. A direct `standard_structured` control on DynaCU-Bench separates the AOI's prompt-format component from its perception component for five models (Section 7.5; Gemini 3 Flash in Section 6.6), ruling out the worst-case confound that the gain is entirely a prompt-format artifact. On a 12-task audio-focused subset, production streaming baselines—OpenAI Realtime (3/12) and Gemini Live (0/12)—underperform AOI (12/12) by a wide margin: streaming voice models with function-calling are not a substitute for a perception layer that persists what was heard.

We release the entire stack—perception layer, DynaCU-Bench, Static-50, DynaCU-Real-Local, and streaming-baseline adapters—so future CU research can cleanly decouple observation gains from model-side gains.

References

Mathieu Andreux et al. Surfer 2: The next generation of cross-platform computer use agents. *arXiv preprint arXiv:2510.19949*, 2025.

Anthropic. Claude computer use. `https://docs.anthropic.com/en/docs/agents-and-tools/computer-use`, 2024a.

Anthropic. Model context protocol. `https://modelcontextprotocol.io`, 2024b.

Ahmed Awadallah et al. Fara-7B: An efficient agentic model for computer use. *arXiv preprint arXiv:2511.19663*, 2025.

Dongping Chen et al. GUI-World: A video benchmark and dataset for multimodal GUI-oriented understanding. *arXiv preprint arXiv:2406.10819*, 2024.

Liang Chen et al. CUA-Suite: 55 hours of real computer-use recordings for agent benchmarking. *arXiv preprint arXiv:2510.10142*, 2025.

Google. Live API (gemini live). `https://ai.google.dev/gemini-api/docs/live`, 2025.

Google DeepMind. Gemini 2.5 Computer Use. https://deepmind.google/models/gemini/computer-use/, 2025.

Google DeepMind. Gemini 3 Flash. https://deepmind.google/models/gemini/, 2026. Accessed 2026-05-15; model id gemini-3-flash-preview.

Hongliang He et al. WebVoyager: Building an end-to-end web agent with large multimodal models. *arXiv preprint arXiv:2401.13919*, 2024.

Lawrence Jang et al. VideoWebArena: Evaluating long context multimodal agents with video understanding web tasks. *arXiv preprint arXiv:2410.19100*, 2024.

Yiqiao Jin et al. ScreenLLM: Stateful screen schema for efficient action understanding and prediction. *arXiv preprint arXiv:2503.20978*, 2025.

Meituan. EvoCUA: Evolving computer use agent. https://github.com/meituan/EvoCUA, 2026. Accessed 2026-05-14.

Microsoft. NLWeb: A conversational interface for websites. https://github.com/microsoft/NLWeb, 2025.

OpenAI. Realtime API: Speech-to-speech models with tool use. https://platform.openai.com/docs/guides/realtime, 2024.

OpenAI. Computer-using agent (CUA). https://openai.com/index/computer-using-agent/, 2025.

OpenAI. Introducing GPT-5.4. https://openai.com/index/introducing-gpt-5-4/, 2026.

Joonsuk Park et al. MemGUI-Bench: Benchmarking memory in GUI agents. *arXiv preprint arXiv:2509.12233*, 2025.

Rui Qian et al. Dispider: Enabling video LLMs with active real-time interaction via disentangled perception, decision, and reaction. *arXiv preprint arXiv:2501.03218*, 2025.

Qwen Team. Qwen3-VL technical report. *arXiv preprint arXiv:2511.21631*, 2025.

Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *ICML*, 2023. arXiv:2212.04356.

Alec Radford et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021. arXiv:2103.00020.

Pascal J. Sager et al. A comprehensive survey of agents for computer use: Foundations, challenges, and future directions. *arXiv preprint arXiv:2501.16150*, 2025.

Simular AI. Agent S3: An open-source computer-use agent. https://github.com/simular-ai/Agent-S, 2025.

Linxin Song et al. CoAct-1: Computer-using agents with coding as actions. *arXiv preprint arXiv:2508.03923*, 2025.

Weihao Tan et al. Cradle: Empowering foundation agents towards general computer control. *arXiv preprint arXiv:2403.03186*, 2024.

Xi Tang et al. Adaptive keyframe sampling for long video understanding. *arXiv preprint arXiv:2502.21271*, 2025.

Haoming Wang et al. UI-TARS-2 technical report: Advancing GUI agent with multi-turn reinforcement learning. *arXiv preprint arXiv:2509.02544*, 2025a.

Xinyuan Wang et al. OpenCUA: Open foundations for computer-use agents. *arXiv preprint arXiv:2508.09123*, 2025b.

Ziyang Wang et al. VideoTree: Adaptive tree-based video representation for LLM reasoning on long videos. *arXiv preprint arXiv:2405.19209*, 2024.

xAI. Grok 4. `https://x.ai/news/grok-4`, 2026. Accessed 2026-05-14.

Tianbao Xie et al. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024. NeurIPS 2024.

Haolin Yang et al. StreamAgent: Towards anticipatory agents for streaming video understanding. *arXiv preprint arXiv:2508.01875*, 2025.

John Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Jiabo Ye et al. Mobile-Agent-v3: Fundamental agents for GUI automation. *arXiv preprint arXiv:2508.15144*, 2025.

Alex L. Zhang et al. VideoGameBench: Can vision-language models complete popular video games? *arXiv preprint arXiv:2505.18134*, 2025.

Ce Zhang et al. A simple LLM framework for long-range video question-answering. *arXiv preprint arXiv:2312.17235*, 2023.

Shuyan Zhou et al. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023. ICLR 2024.

Orr Zohar et al. Apollo: An exploration of video understanding in large multimodal models. *arXiv preprint arXiv:2412.10360*, 2024.

A Full Per-Category Results

Table 20: Complete per-category results for all 8 observation configurations tested with Claude Sonnet 4.6.

Mode	A	B	C	D	E	F	G	H	I	J	Total
Standard	0	2	4	4	8	4	1	7	3	5	38
Uniform 1 FPS	0	4	10	8	9	8	1	8	6	4	58
Uniform 3 FPS	2	5	8	8	8	8	1	7	6	3	56
Pixel-diff	1	5	9	7	9	8	1	8	6	4	58
Random keyframes	1	6	8	6	9	8	1	8	6	3	56
AOI visual	1	5	9	7	8	9	1	8	6	4	58
AOI visual+ASR	1	4	10	9	9	9	4	9	6	3	64
AOI full	**9**	**10**	**9**	**9**	**8**	**9**	**9**	**9**	**8**	2	**82**

B DynaCU-Bench Task List

Each of the 100 tasks is identified by a category letter (A–J) and a difficulty-indexed ID. Easy tasks: E1, E2, E3. Medium tasks: M1, M2, M3, M4. Hard tasks: H1, H2, H3.

Example tasks by category:

- **A-E1** (Podcast, Easy): Listen to a podcast segment and identify the guest's name.
- **B-M2** (Meeting, Medium): During a meeting with slides, count the number of items in a presented chart.
- **C-H1** (Video, Hard): Watch a terminal recording and describe the full workflow demonstrated.
- **D-M4** (Carousel, Medium): Identify a specific product name from a rotating carousel.
- **E-E3** (Dashboard, Easy): Read the current warning level from a live-updating dashboard.
- **F-E2** (Transient UI, Easy): Click "Accept" on a cookie consent banner before it auto-dismisses.
- **G-M1** (Phone, Medium): Follow spoken instructions during a simulated phone call.
- **H-H2** (Interview, Hard): Complete a multi-question spoken interview with scoring.
- **I-M2** (Collab, Medium): Resolve a conflict in a collaborative document when a remote edit arrives.
- **J-E2** (Games, Easy): Complete 3 rounds of a simple reaction-time game.

All tasks are implemented as self-contained HTML files with embedded JavaScript for task logic, audio synthesis, and evaluation. The complete benchmark (100 HTML files, evaluation scripts, and task definitions) will be released publicly.

C CLIP Threshold Calibration

The default threshold $\theta = 0.04$ was calibrated empirically. Web UI events such as modal dialogs appearing produce cosine distances of $\sim$0.08; video scene cuts produce distances of $\sim$0.15–0.25; loading spinners and cursor blinks produce distances of $\sim$0.005–0.02. Setting $\theta = 0.04$ captures all meaningful UI changes while filtering most periodic noise.

As shown in Figure 7, the method is robust across a wide range: any $\theta \in [0.02, 0.30]$ produces success rates within an 80–90% point-estimate band (CIs overlap, so individual thresholds are not statistically distinguishable at N=40). This insensitivity arises because CLIP embeddings produce a

Table 21: DynaCU-Real-Local: Claude Sonnet 4.6 Standard vs. AOI full on 12 real-recording tasks. Both modes tie at 11/12 on this set; the failure is the same task (`R_cast2_pip`) for both.

Mode	Podcast	Meeting	Screencast	Voice	Total / 12
Standard	3/3	3/3	2/3	3/3	**11** (92%)
AOI full	3/3	3/3	2/3	3/3	**11** (92%)

bimodal distribution of distances: semantically unchanged frames cluster near zero, while meaningful changes produce large distances, with relatively few samples in between.

D Implementation Details

Software. Python 3.11, Playwright 1.49 for browser automation, PulseAudio 16.1 for audio I/O, edge-TTS for high-quality speech synthesis, OpenAI CLIP ViT-B/16 for keyframe extraction, Whisper large-v3 for speech transcription, vLLM 0.19.0 for local model serving.

Hardware. All experiments run on a single machine with an NVIDIA RTX PRO 6000 Blackwell GPU (96 GB VRAM), 192 GB RAM. Cloud models (Claude, GPT-5.4, Gemini 2.5 Flash) are accessed via HTTP APIs. Local models (EvoCUA-32B, Fara-7B) are served via vLLM with 4-bit quantization (`bitsandbytes`). Whisper runs on CPU to avoid GPU contention.

Hyperparameters. Screen capture rate: ∼3 Hz. Pixel change threshold (α): 1%. CLIP distance threshold (θ): 0.04. Maximum keyframes per step: 5. Audio capture: a 70 s ring buffer at 16 kHz mono holds rolling history; Whisper is invoked on a ∼3.5 s segment per step (with a ∼3.5 s overlap into the previous segment for boundary continuity) when the volume gate fires. Post-action wait: 2.0 s. Maximum steps per task: 15.

E DynaCU-Real-Local Details

The 12 tasks span four sub-domains: 3 podcast (Aesop animal-identification), 3 meeting (Python / Postgres / Rust technical detail), 3 screencast (`git clone` / `pip install` / `npm test` outcome), and 3 voice (yes/no / directions / appointment day). Audio for podcast/meeting/voice tasks is rendered with `espeak` (a formant synthesizer with a deliberately non-neural acoustic profile). Screencast tasks use real `asciinema` v2 cast files generated locally. Source texts are public-domain materials: Aesop's Fables for podcasts, Wikipedia for Python language facts, project documentation for PostgreSQL/MVCC and Rust/tokio. All HTML harnesses, asset-build scripts, and success checks are released with the benchmark.

F Streaming-Baseline Adapter Sanity Check Details

To rule out the explanation that the streaming baselines' weak audio-subset numbers (Section 7.6) are an artifact of our adapter rather than a model limitation, we evaluate both adapters on a small purely-visual sanity set of five tasks across four categories: C (video / static recording), E (live dashboard), F (transient UI; two tasks, one banner-accept and one cookie-accept subtype), and I (collaborative document). All chosen so the answer can be reached without any audio comprehension. The exact set is C-E1, E-E1, F-E1, F-E2, and I-E1. For each baseline, we open one streaming session per task, send the post-action screenshot as a multimodal turn, and translate any returned tool calls into browser actions through the same path used in the main audio-subset evaluation.

Table 22: Per-task adapter sanity-check outcomes for each streaming baseline. Both adapters drive the browser and emit valid tool calls (`click`, `fill`, `type_text`) on every task; every attempted action lands on the DOM. Both score 0/5 task-correct because the models choose the wrong target (e.g., F-E2 cookies dismissed instead of accepted), hallucinate content (C-E1 typed a URL not present in the recording), or commit prematurely (I-E1 typed before reading the page). The same failure modes appear on the audio subset (Table 18), confirming the audio-subset failures are content-driven rather than infrastructural.

Baseline	**C-E1**	**E-E1**	**F-E1**	**F-E2**	**I-E1**	**Total**
OpenAI Realtime (gpt-4o + vision tools)	✗	✗	✗	✗	✗	0
Gemini Live (2.5 flash native-audio)	✗	✗	✗	✗	✗	0
AOI full (Claude Sonnet 4.6) reference	✓	✗	✓	✓	✓	4

The success criterion is the same DOM check as the main benchmark. Table 22 reports the per-task outcome and the modal failure mode where applicable.